Adaptive CPU Scheduling Using Learned Burst-Length Prediction

Shivaansh Aggarwal
Dept. of Computer Science and Engineering
[Institution name]
[City], India
Reg. No. 24BCE0883

Devansh Sharma
Dept. of Computer Science and Engineering
[Institution name]
[City], India
Reg. No. 24BCE0717

***Abstract*—Classical CPU schedulers are static: first-come-first-served ignores burst length, shortest-job-first needs the length of the next CPU burst before it happens, and round robin uses a time quantum chosen once at configuration time. Operating systems usually estimate the next burst with exponential averaging, which lags whenever a process changes behaviour. We present an adaptive scheduler that predicts each process's next CPU burst with a gradient-boosted regressor over twelve history features, and uses the prediction twice: to order the ready queue with an ageing term that prevents starvation, and to recompute the time quantum continuously as an error-aware percentile of the predicted bursts. Using a verified discrete-event simulator, we compare eleven scheduler configurations on six workload types over ten paired seeds (660 runs). The learned predictor is 12–22% more accurate than exponential averaging on every workload. On a mixed workload the scheduler reduces average waiting time by 27% relative to SJF with exponential averaging, better on all ten seeds. It closes 54% of the gap to an oracle with perfect knowledge, lowers worst-case waiting by 23%, and needs 61% fewer context switches than a 5 ms round robin. It does not help on uniform workloads of short random bursts, and we report these negative results alongside an ablation of every component.**

***Index Terms*—CPU scheduling, burst prediction, machine learning, time quantum, ageing, discrete-event simulation, operating systems.**

I. Introduction

The short-term scheduler decides which ready process receives the CPU next and for how long. That decision, repeated thousands of times per second, determines waiting time, responsiveness and the overhead lost to context switching. The policies taught in every operating systems course [1]–[3] are static. First-come-first-served (FCFS) suffers from the convoy effect: with bursts of 24, 3 and 3 ms the average wait is 17 ms. Shortest-job-first (SJF) is optimal for average waiting time (3 ms on the same jobs), but it requires the length of the next CPU burst, which does not exist when the decision must be made. Round robin (RR) bounds response time with a fixed quantum: small quanta cause many context switches, large quanta make the system sluggish.

Real systems approximate SJF by estimating the next burst with exponential averaging, $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$ [1]. The estimate uses only one number, the process's own past burst lengths, and lags after a phase change. For bursts 2, 2, 2, 2, 60, 60, 60 with $\alpha = 0.5$ it predicts 2, 31, 45.5 and 52.75, misjudging the process for three to four bursts. Similarly, the textbook guideline that about 80% of CPU bursts should fit within one quantum [1] is applied once by a person, because no scheduler measures the burst distribution.

This paper asks whether a learned burst predictor, used both for ordering and for quantum sizing, measurably improves scheduling. Our contributions are:

- a single-CPU discrete-event simulator, verified against textbook results, in which every policy sees identical workloads;
- a burst predictor trained and evaluated with a split by process, compared directly with exponential averaging;
- an adaptive scheduler combining predicted-burst ordering, ageing, and an error-aware quantum controller;
- a paired evaluation of 660 runs with hypotheses stated in advance, an ablation study, and an explicit report of where the method does not help.

II. Background and Related Work

FCFS, SJF, shortest-remaining-time-first (SRTF) and RR are standard [1], [2]. The multilevel feedback queue (MLFQ), introduced with the Compatible Time-Sharing System [7], approximates SJF without burst estimates by demoting processes that consume their quantum and periodically boosting all processes to prevent starvation [3]. Linux's Completely Fair Scheduler instead allocates CPU time in proportion to weights and does not predict bursts [6]. Predicting run times from history has a long record in batch and parallel job scheduling, where historical information [8] and system-generated predictions [9] improve on user estimates. Our work applies the same idea at the granularity of individual CPU bursts. We compare against an oracle that knows every burst, which shows how much of the achievable improvement a predictor captures. We use gradient boosting [10] through scikit-learn [5], and Jain's index [4] for fairness.

III. System Design

A. Simulation Engine

The engine is event-driven: a binary-heap queue holds ARRIVAL, IO_DONE and SLICE_END events, and all events at the same instant are processed before the next decision. Each process alternates CPU bursts and I/O waits. The engine owns the clock and the process control blocks but makes no decisions: it calls a policy interface (`select`, `time_slice`, `should_preempt` and event hooks). All policies therefore see

identical arrivals, bursts, I/O times and a 0.1 ms context-switch cost, charged whenever the CPU moves to a different process. Only the engine and the oracle can read true remaining burst times.

B. Workload Model

Four process types are generated (Table I). Arrivals are Poisson, with the rate set so the offered CPU load is 85%. Six profiles of 200 processes are used: W1 CPU-bound, W2 I/O-bound, W3 interactive, W4 mixed (40% interactive, 30% CPU-bound, 20% I/O-bound, 10% phase-changing), W5 phase-changing, and W6, which is W4 at three times the arrival rate (overload).

TABLE I
PROCESS TYPES (TIMES IN MILLISECONDS)

Type	CPU burst	Mean I/O	Bursts
CPU-bound	lognormal, median 100	5	3–8
I/O-bound	exponential, mean 2	30	20–60
Interactive	exponential, mean 5	100	10–40
Phase-changing	I/O-bound, then CPU-bound	—	10–30

C. Burst Predictor

Each process is described by twelve features, all of which a kernel already tracks: the last five burst lengths, the mean and standard deviation of past bursts, the exponential-average estimate itself (so the model can only add to it), the last and mean I/O wait, the number of completed bursts, and the last change in burst length. Features use only completed bursts, and a unit test confirms that changing a process's future does not change its features. Because features depend only on a process's own history, training data are built directly from 2,000 generated processes (45,389 bursts) from seeds never used in evaluation, split 75/25 *by process*. Selection rule: the lowest held-out mean absolute error (MAE) within a 1 ms latency budget, with ridge regression preferred if within 5%. Table II shows that gradient boosting on log(burst) wins. A logistic-regression classifier (macro-F1 0.94, against 0.94 for a random forest at 20× the latency) labels processes as CPU-bound, I/O-bound or interactive.

TABLE II
HELD-OUT PREDICTION ERROR (MS) BY PROCESS TYPE

Model	MAE	CPU	I/O	Inter.	Phase
Exponential avg.	12.36	60.1	1.91	4.38	31.5
Ridge	14.82	54.9	5.89	5.86	34.2
Ridge (log)	22.54	96.4	1.58	3.58	70.9
Grad. boosting	12.59	53.1	3.59	4.76	30.5
Grad. boost. (log)	**10.62**	**54.1**	**1.53**	**3.59**	**26.8**

A feedback loop compares every completed burst with its prediction, logs the error and stores the example in a rolling buffer. Every 2,000 new examples the model is refit on the offline data plus the buffer.

D. Adaptive Scheduler

Let $\hat{b}_i$ be the predicted length of process *i*'s current burst, d_i the CPU time it has already received in that burst, and w_i the time it has waited since entering the ready queue. The estimated remaining time is $r_i = \hat{b}_i - d_i$, or $\hat{b}_i$ if the burst has already outrun its prediction. The process with the lowest priority value runs next:

$$P_i = r_i - \beta\, w_i \tag{1}$$

With $\beta = 0$ this is exactly SJF on predicted bursts (verified slice by slice in a unit test). The ageing term guarantees that a long process eventually reaches the front. The value $\beta = 0.03$ was chosen on tuning seeds that are never used in evaluation. Every $E = 50$ decisions the quantum is recomputed from the predictions $\hat{B}$ of bursts that became ready during the epoch, and the last 200 ratios ρ = actual/predicted:

$$q_{new} = clamp(P_p\{\, \hat{b}\,\rho : \hat{b} \in \hat{B},\, \rho \in R \,\},\ 1,\ 100) \tag{2}$$

$$q \leftarrow \gamma\, q_{new} + (1 - \gamma)\, q \tag{3}$$

with $p = 80$ and $\gamma = 0.3$. The ratios matter. A prediction is a conditional average, so predictions are far less dispersed than real bursts: for I/O-bound bursts with mean 2 ms, nearly every prediction is about 2 ms, yet 20% of real bursts exceed 3.2 ms. A first version that used raw predictions produced quanta that were too small on W2 and W3. Multiplying by observed error ratios restores the real spread, so the textbook 80% rule is applied to an estimate of the true burst distribution. Finally, the percentile is shifted by 10 × (fraction CPU-bound − fraction interactive) using the classifier, which favours responsiveness for interactive mixes and fewer switches for CPU-bound ones.

E. Baselines

The eleven configurations are: FCFS; SJF and preemptive SRTF, both with exponential averaging ($\alpha = 0.5$, $\tau_0 = 10$ ms); RR with quanta of 2, 5, 10 and 20 ms, so the proposal cannot win against a badly tuned quantum; MLFQ with three levels (4/8/16 ms) and a boost every 200 ms; an oracle, SRTF with true burst lengths; and our scheduler with exponential averaging (Adaptive-EMA) and with the learned model (Adaptive-ML). Adaptive-EMA isolates the contribution of the learned predictor from that of the scheduling mechanism.

IV. EXPERIMENTAL METHODOLOGY

Each of the 6 workloads is generated with 10 seeds, and every configuration runs on the same 60 instances (660 paired runs). For process *i* with arrival a_i, completion c_i, first dispatch s_i, total CPU demand B_i and I/O time IO_i:

$$T_i = c_i - a_i,\ \ W_i = T_i - B_i - IO_i,\ \ R_i = s_i - a_i \tag{4}$$

We report average and worst-case W, response R, context switches, utilisation, Jain's index [4] on each process's progress rate (B + IO)/T, the number of starved processes (W above five times the cross-policy mean), prediction MAE, and the share of the SJF-EMA-to-oracle gap that is closed, $G = (W_{SJF} - W_{ours})/(W_{SJF} - W_{oracle})$. Seven hypotheses were stated before any experiment (Table IV). An improvement counts only if the mean paired improvement over seeds exceeds one standard deviation of the paired differences. Additional runs comprise an ablation on W4 and W5 (120

runs), sweeps of β, p and E on W4 (130 runs) and switch costs of 0.05, 0.1 and 0.5 ms (150 runs).

V. Results

A. Simulator Verification

Before any learning, the engine was checked against hand-computed cases [1]: FCFS on bursts 24, 3, 3 gives exactly 17 ms, shortest-first 3 ms, RR with q = 4 gives 5.67 ms, and SRTF on bursts 8, 4, 9, 5 arriving at 0, 1, 2, 3 gives 6.5 ms (Fig. 1). Total CPU service is identical across all eleven policies, event times are monotonic, and all 37 unit tests pass.

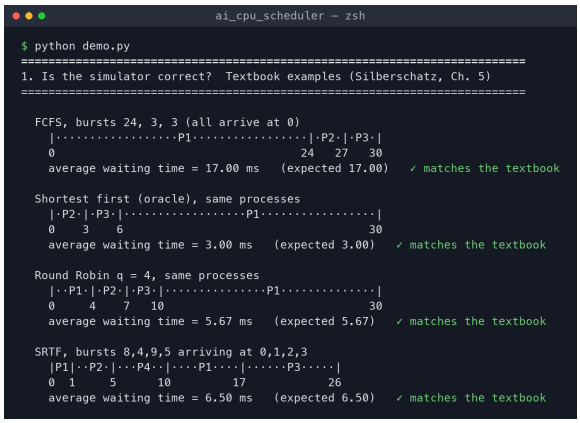


Fig. 1. Output of the demonstration program: Gantt charts and average waiting times for the textbook examples, each matching the expected value exactly.

B. Waiting Time

Table III and Fig. 2 give average waiting time for every configuration. On the mixed workload W4, Adaptive-ML waits 1,786 ms against 2,460 ms for SJF-EMA, a 27.4% reduction. It is better on all ten seeds (mean paired improvement 674 ms, standard deviation 117 ms) and closes 54% of the gap to the oracle. On W1 and W5 the average is within 2% of SJF-EMA, and on W2, W3 and W6 it is 14%, 5% and 3% higher. Adaptive-ML beats Adaptive-EMA on four of six workloads (W1, W4, W5, W6).

Table III
Average Waiting Time (ms), Mean ± Standard Deviation over 10 Seeds

Scheduler	W1 CPU	W2 I/O	W3 interactive	W4 mixed	W5 phase	W6 overload
FCFS	2,356 ± 1,467	454 ± 194	490 ± 155	4,003 ± 1,780	4,330 ± 1,433	38,210 ± 1,937
SJF-EMA	2,778 ± 1,727	395 ± 143	480 ± 147	2,460 ± 791	6,269 ± 2,451	17,150 ± 1,801
SRTF-EMA	2,881 ± 1,728	432 ± 154	489 ± 159	1,729 ± 720	6,830 ± 2,320	16,490 ± 1,747
RR-2	3,478 ± 2,021	556 ± 245	608 ± 213	2,394 ± 1,069	7,675 ± 2,463	22,998 ± 2,097
RR-5	2,931 ± 1,753	470 ± 203	523 ± 180	2,180 ± 989	6,576 ± 2,266	23,615 ± 2,096
RR-10	2,755 ± 1,668	460 ± 198	498 ± 166	2,276 ± 1,047	6,127 ± 2,131	26,088 ± 2,204
RR-20	2,661 ± 1,624	454 ± 194	488 ± 156	2,572 ± 1,210	5,713 ± 1,986	30,333 ± 2,389
MLFQ	2,814 ± 1,733	503 ± 215	542 ± 187	1,878 ± 869	6,566 ± 2,340	22,762 ± 2,167
Oracle	1,697 ± 930	370 ± 200	283 ± 113	1,204 ± 484	4,617 ± 1,590	12,820 ± 1,542
Adaptive-EMA	2,915 ± 1,791	423 ± 169	487 ± 159	1,851 ± 752	6,569 ± 2,436	18,017 ± 1,914
Adaptive-ML	**2,725 ± 1,692**	**450 ± 180**	**505 ± 182**	**1,786 ± 730**	**6,163 ± 2,235**	**17,599 ± 2,335**

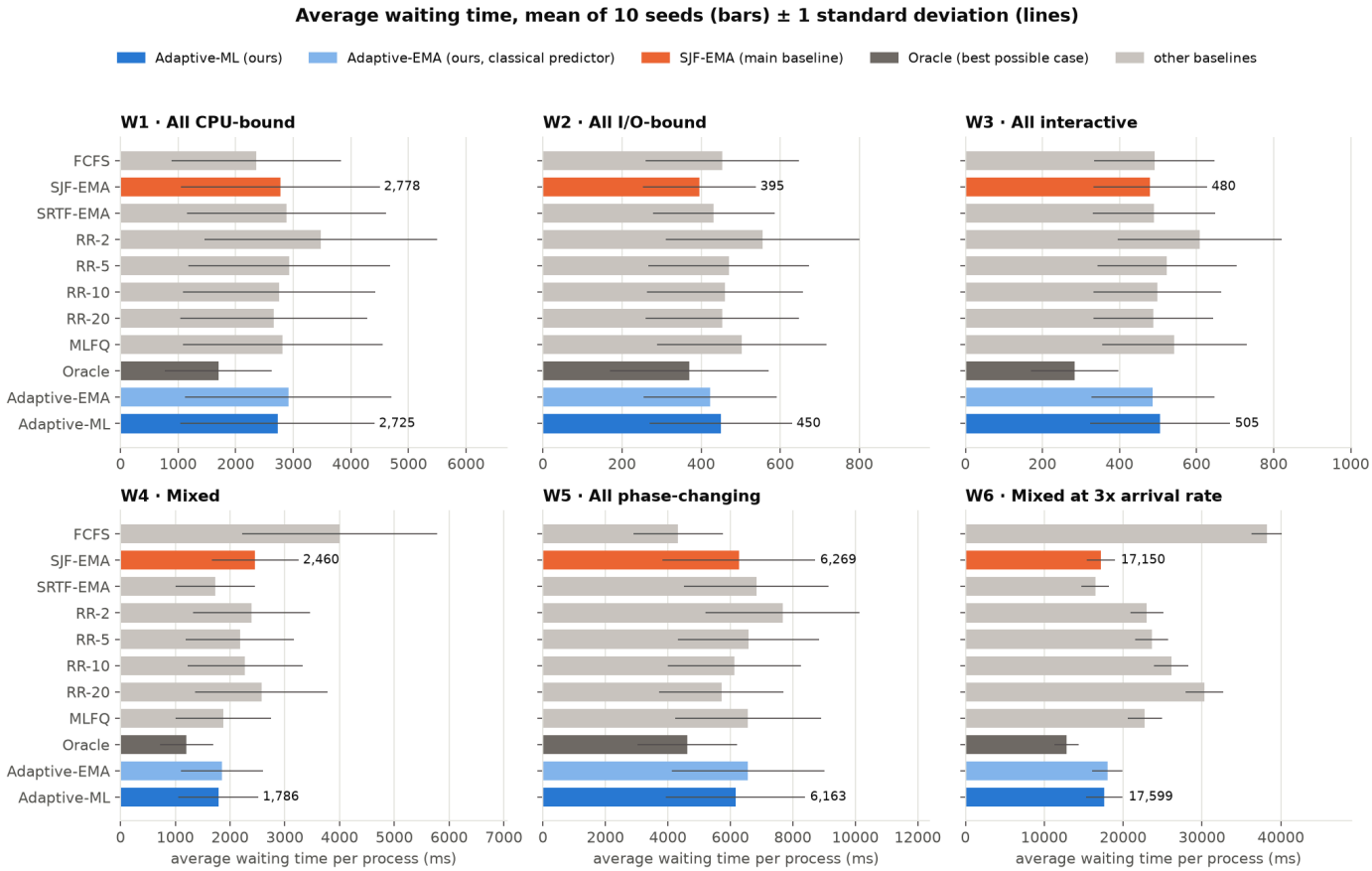


Fig. 2. Average waiting time per workload (bars: mean of 10 seeds; lines: ±1 standard deviation). Adaptive-ML is blue, the main baseline SJF-EMA orange, and the oracle dark grey.

C. Worst Case, Switches and Response

Ageing bounds starvation. Adaptive-ML's worst-case wait is lower than SJF-EMA's by 56% on W1, 27% on W3, 23% on W4 and 53% on W5, and lower than SRTF-EMA's on five of six workloads (not W2). On W5 the number of starved processes per run falls from 7.7 (SJF-EMA) to 0.2. On W4, Adaptive-ML performs 5,933 context switches against 15,328 for RR-5 (−61%) and 34,354 for RR-2, with an average response time of 15.4 ms, against 23.3 ms for RR-5 and 69.3 ms for RR-20. The cost is more switches than non-preemptive SJF-EMA (3,977). When the switch cost rises from 0.1 to 0.5 ms, Adaptive-ML's average wait on W4 grows by 20%, against 53% for MLFQ and 62% for RR-5.

D. Adaptation of the Quantum

Fig. 3 runs Adaptive-ML on a workload whose mix changes every 30 s. The quantum settles near 6 ms while interactive processes arrive, rises to the 100 ms cap for CPU-bound processes, and falls to about 3.5 ms for I/O-bound ones, close to the analytical 80th percentile of an exponential with mean 2 ms (3.2 ms). No fixed quantum suits all three phases. Across the main workloads the mean quantum was 92 ms (W1), 3.5 ms (W2), 6.4 ms (W3), 18 ms (W4), 95 ms (W5) and 14 ms (W6).

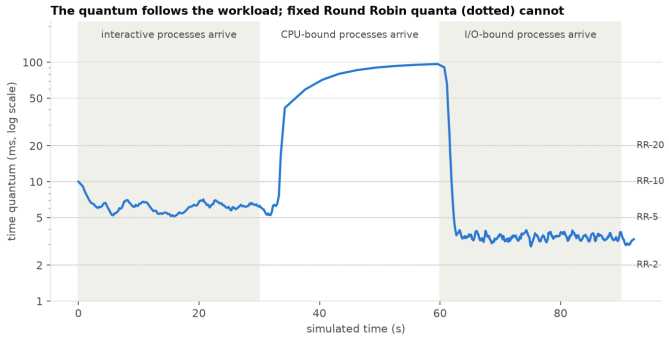


Fig. 3. Time quantum of Adaptive-ML on a workload that changes from interactive to CPU-bound to I/O-bound processes. Dotted lines show fixed RR quanta.

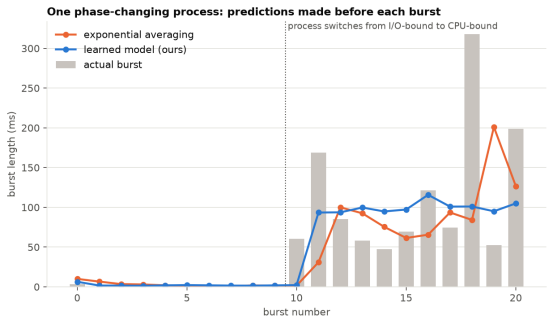


Fig. 4. Predictions for one phase-changing process. After the first long burst the learned model jumps to about 95 ms, while exponential averaging predicts 31 ms.

E. Hypotheses and Ablation

Table IV lists the verdicts. Table V switches off one component at a time. Removing ageing lowers the W4 average by 8% but raises its worst case by 26%; on W5 it leaves the average unchanged and doubles the worst case. Fixing the quantum at 10 ms barely changes W4 but raises the W5 average by 11% and worst case by 52%. Using the learned prediction alone in non-preemptive SJF ("prediction only") improves on SJF-EMA by 5% on W4. Removing class modulation or online refitting changes the average wait by less than 3%.

TABLE IV
HYPOTHESES STATED BEFORE THE EXPERIMENTS

Hypothesis	Verdict
H1 Lower avg. wait than SJF-EMA on W4–W6	Partly (W4 only)
H2 Far fewer switches than small-q RR	Supported
H3 Lower worst wait than SJF/SRTF-EMA	Partly (5 of 6)
H4 ≈ SJF-EMA on W1, W2 (±10%)	Partly (W1 only)
H5 Never beats the oracle	Supported (0/60)
H6 Lower MAE than exp. averaging	Supported (6 of 6)
H7 No ageing: lower avg., higher worst	Partly (W4 only)

TABLE V
ABLATION OF ADAPTIVE-ML (MS, MEAN OF 10 SEEDS)

Variant	W4 avg	W4 worst	W5 avg	W5 worst
Full	**1,786**	**18,388**	**6,163**	**25,831**
No ageing	1,643	23,182	6,196	53,915
Fixed quantum	1,763	20,138	6,864	39,254
No class modulation	1,807	18,898	6,238	27,875
No online refit	1,833	17,766	6,134	25,482
Prediction only	2,336	22,759	6,325	53,593
SJF-EMA	2,460	23,766	6,269	55,480

VI. DISCUSSION

Where it helps. The gains appear where there is heterogeneity to exploit: the mixed workload, where short interactive bursts compete with long CPU-bound ones. Most of the benefit comes from *how* the prediction is used (ordering, ageing and quantum together). Better prediction accuracy alone yields only a modest gain.

Where it does not. On W2 and W3 the bursts are short and exponentially distributed, so they are essentially unpredictable beyond their mean, and preempting at a 3–6 ms quantum adds switching overhead without improving the order. Under overload (W6) waits reach tens of seconds, so the linear ageing term dominates and the order drifts towards FCFS. A wait-normalised ageing term is a natural refinement.

The oracle is not a lower bound. SRTF with true burst lengths is optimal for single-burst jobs, but not when processes have many bursts. On W5 plain FCFS (4,330 ms) beats the oracle (4,617 ms), because serving the shortest burst first repeatedly starves processes in their long phase. The oracle-gap metric should therefore be read as a reference for burst ordering, not as an absolute floor.

Limitations. The study is simulation-only, with one CPU and no cache or memory effects. Workloads are synthetic, and the model is trained on the same generator (with disjoint seeds and processes), so it says nothing about distribution shift. This is also why online refitting showed no benefit. A prediction takes about 0.5 ms in Python, far above what a kernel can afford. A deployable version would need a much smaller model, such as a shallow tree or lookup table.

VII. CONCLUSION

We showed that using a learned burst prediction twice, to order the ready queue with ageing and to size the quantum from an error-aware percentile of predicted bursts, reduces average waiting time on a mixed workload by 27% relative to SJF with exponential averaging. It also cuts worst-case waiting and context switches, while performing no better, or slightly worse, on uniform workloads of short random bursts. Future work will evaluate the scheduler on traced real workloads and port the quantum controller to a teaching kernel such as xv6. The simulator, models, experiment scripts and all raw results are included with the project for reproduction.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018, ch. 5.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2014.

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.00. Arpaci-Dusseau Books, 2018, chs. 7–8.

[4] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] Linux Kernel Documentation, "CFS scheduler." [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html

[7] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, "An experimental time-sharing system," in *Proc. Spring Joint Comput. Conf. (AFIPS)*, 1962, pp. 335–344.

[8] W. Smith, I. Foster, and V. Taylor, "Predicting application run times using historical information," in *Job Scheduling Strategies for Parallel Processing*, LNCS vol. 1459. Berlin, Germany: Springer, 1998, pp. 122–142.

[9] D. Tsafrir, Y. Etsion, and D. G. Feitelson, "Backfilling using system-generated predictions rather than user runtime estimates," *IEEE Trans. Parallel Distrib. Syst.*, vol. 18, no. 6, pp. 789–803, Jun. 2007.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.